UNIVERSITY OF LIMPOPO

FACULTY OF SCIENCE AND AGRICULTURE

SCHOOL OF MATHEMATICAL AND COMPUTER SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

DEGREE AND DIPLOMA PROJECT 1 **Due date: 09 OCT 2026**

MODULE: SCSB082 **PAPER: P**

(ADVANCED OPERATING SYSTEMS)

TIME: **3:00 HOURS** **MARKS: 100**

INTERNAL EXAMINER: **DR J TLOUYAMMA**

SECOND EXAMINER: **PROF M VELEMPINI**

THIS PAPER CONSISTS OF (19) PAGES INCLUDING COVER PAGE

INSTRUCTIONS:

1. Project Type: Group Project
2. Submission: Working simulation + technical report + demonstration
3. Recommended Group Size: 3–4 students
4. Programming/Simulation Environment: Students may use any appropriate environment, such as Python, Java, C#, C++, JavaScript/HTML, MATLAB/Simulink, or another approved simulation platform.

Project Title

Design and Development of an Operating Systems Resource Management Simulator

2. Project Background

Modern computer systems must efficiently manage processes, main memory, virtual memory, secondary storage, and storage redundancy.

In this project, students are required to design and implement a **simulation of an operating system environment** that demonstrates how processes are executed and how the operating system manages memory and storage resources.

The simulator must demonstrate the interaction between:

Processes → CPU → Main Memory → Virtual Memory → Secondary Storage → RAID Storage

The purpose is not to build a real operating system. Instead, students must develop a simulation that models the behaviour of an operating system and allows a user to observe what happens when processes are loaded, executed, suspended, moved between memory levels, and when storage operations occur.

3. Project Scenario

A university research centre is developing a high-performance computing environment for processing large datasets.

The system has the following characteristics:

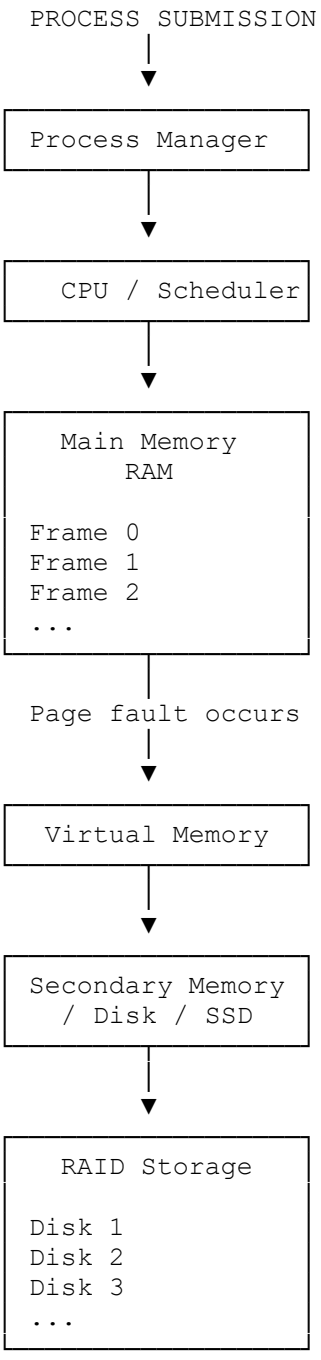
- Multiple processes compete for CPU time.
- Main memory is limited.
- Processes are divided into pages.
- Not all pages can remain in the main memory simultaneously.
- Pages may need to be transferred between main memory and secondary storage.
- Virtual memory is used when physical memory is insufficient.
- Different page replacement algorithms may produce different performances.
- The storage subsystem must provide improved reliability and performance using RAID.

The research center has asked your team to develop a simulation tool that demonstrates how the operating system manages these resources.

The simulator must allow an administrator or student to observe the movement of processes and data through the system.

4. Overall System Architecture

Your simulator should represent an environment similar to the following:



Students may implement the architecture differently, provided that the required functionality is demonstrated.

5. Project Objectives

At the end of the project, students should be able to:

1. Demonstrate how an operating system manages processes.
2. Simulate CPU execution of multiple processes.
3. Demonstrate the allocation of processes to main memory.
4. Divide processes into pages.
5. Demonstrate page allocation to physical memory frames.
6. Simulate page faults.
7. Demonstrate movement of pages between RAM and secondary storage.
8. Implement virtual memory concepts.
9. Implement and compare page replacement algorithms.
10. Demonstrate the effect of different memory sizes.
11. Simulate RAID configurations.
12. Compare RAID levels in terms of performance, storage efficiency and fault tolerance.
13. Analyse the effect of resource constraints on system performance.
14. Design a user interface that allows users to observe the simulation.
15. Collect and analyse performance statistics.

6. Required Simulation Components

PART A – PROCESS EXECUTION AND CPU MANAGEMENT

20 Marks

The first component must simulate how processes are submitted to and executed by the CPU.

Students must create a minimum of **5 processes**.

Each process should have at minimum:

- Process ID
- Arrival time
- CPU burst time
- Priority
- Memory requirement
- Process size
- Number of pages
- Process status

Example:

Process	Arrival Time	Burst Time	Priority	Size
P1	0	8	2	12 KB
P2	1	5	1	8 KB
P3	2	12	3	20 KB

Process	Arrival Time	Burst Time	Priority	Size
P4	3	6	2	16 KB
P5	4	10	4	24 KB

The simulator should show process states such as:

```
New
 ↓
Ready
 ↓
Running
 ↓
Waiting/Blocked
 ↓
Ready
 ↓
Terminated
```

Required features

The simulator must:

- Create processes.
- Place processes in a ready queue.
- Select processes for CPU execution.
- Simulate execution.
- Display the currently running process.
- Display completed processes.
- Display waiting processes.
- Show CPU utilisation.
- Display waiting time and turnaround time.

Students should implement at least **two CPU scheduling algorithms**, for example:

- First-Come, First-Served (FCFS)
- Round Robin
- Shortest Job First
- Priority Scheduling

Students must compare the algorithms.

Expected output

The simulator should produce something similar to:

```
Time: 15

CPU
```

```
Running: P3
Burst: 12
Remaining: 7
```

```
Ready Queue:
P4 → P5 → P1

Completed:
P2
```

A Gantt chart should also be generated.

7. PART B – PAGING AND MAIN MEMORY

25 Marks

The second component must demonstrate how processes are divided into pages and loaded into physical memory.

Assume, for example:

```
Main Memory = 32 KB
Page Size   = 4 KB
```

Therefore:

```
Number of Frames = 32 KB / 4 KB
                 = 8 frames
```

A process of 14 KB would require:

```
Number of Pages = Ceiling(14 / 4)
                = 4 pages
```

The simulator must show:

```
Process P1

Page 0 ⟶ Frame 3
Page 1 ⟶ Frame 7
Page 2 ⟶ Frame 1
Page 3 ⟶ Frame 5
```

Required functionality

Students must implement:

- Configurable RAM size.
- Configurable page size.
- Calculation of number of frames.
- Process-to-page conversion.
- Page-to-frame allocation.
- Page table.
- Free-frame list.
- Occupied-frame list.

The page table should contain information similar to:

Page	Frame	Valid Bit	Location
0	3	1	RAM
1	7	1	RAM
2	-	0	Disk
3	5	1	RAM

The simulator should visually distinguish between:

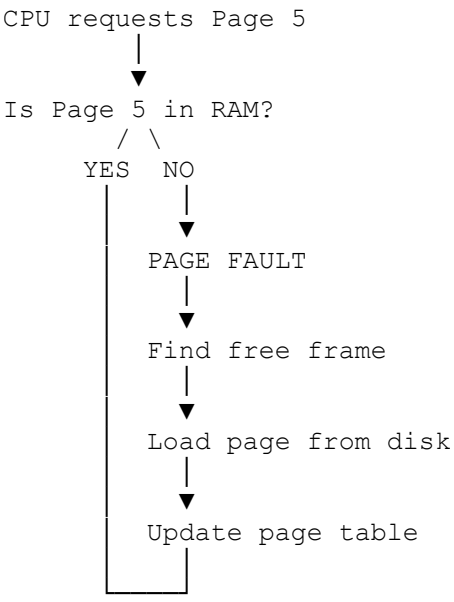
- Pages are currently in RAM.
- Pages currently on secondary storage.
- Free frames.

8. PART C – PAGE FAULTS AND MOVEMENT BETWEEN MEMORY LEVELS

15 Marks

The simulator must demonstrate what happens when a process references a page that is not currently located in main memory.

For example:



If there is no free frame, the simulator must select a page for replacement.

The system should clearly display:

```
PAGE FAULT!
```

```
Requested Page: P3
Page: 5

Action:
1. Locate page on secondary storage
2. Select victim page
3. Move victim page to secondary storage
4. Load requested page into RAM
5. Update page table
6. Continue process execution
```

Students should record:

- Number of page faults.
- Number of page hits.
- Page fault rate.
- Number of pages transferred.
- Number of page replacements.

9. PART D – VIRTUAL MEMORY AND PAGE REPLACEMENT

20 Marks

Students must demonstrate how virtual memory allows a process to use an address space larger than the available physical memory.

For example:

```
Virtual Memory = 64 KB
Physical Memory = 16 KB
Page Size = 4 KB
```

Therefore:

```
Virtual Pages = 64 / 4
              = 16 pages

Physical Frames = 16 / 4
                = 4 frames
```

The simulator should demonstrate that a process can have 16 virtual pages while only 4 pages can be resident in physical memory at a time.

Page Replacement Algorithms

Students must implement at least **three**-page replacement algorithms:

1. FIFO

The oldest page currently in memory is replaced.

2. LRU

The page that has not been used for the longest period is replaced.

3. Optimal

The page that will not be used for the longest period in the future is replaced.

The simulator must allow the user to select an algorithm.

Example:

```
Page Reference String:

7 0 1 2 0 3 0 4 2 3 0 3
```

The simulator should show:

Reference	Frame 1	Frame 2	Frame 3	Fault
7	7	-	-	Yes
0	7	0	-	Yes
1	7	0	1	Yes
2	2	0	1	Yes
0	2	0	1	No
3	2	0	3	Yes

The final results should include:

```
Algorithm: FIFO

Page References: 100
Page Hits:       62
Page Faults:     38
Hit Rate:        62%
Fault Rate:      38%
```

Students must compare the performance of the three algorithms.

10. PART E – RAID TECHNOLOGY

15 Marks

The final component must simulate storage using RAID.

The university currently uses a **low-level RAID configuration that provides poor performance for the workload**.

Students must investigate and simulate alternative RAID configurations.

At least **three RAID levels** must be considered, for example:

- RAID 0
- RAID 1
- RAID 5
- RAID 6
- RAID 10

Students should simulate how data is distributed across multiple disks.

For example, RAID 0:

```
Data: A B C D E F

Disk 1: A     C     E
Disk 2:   B     D     F
```

RAID 1:

```
Disk 1: A B C D
Disk 2: A B C D
```

RAID 5:

```
Disk 1: A   B   P
Disk 2: C   P   E
Disk 3: P   D   F
```

The exact implementation may vary.

11. RAID Failure Simulation

The simulator must allow at least **one disk failure** to occur.

For example:

```
RAID 5

Disk 1: ONLINE
Disk 2: FAILED
Disk 3: ONLINE
Disk 4: ONLINE
```

Students must demonstrate whether the data remains available.

The simulator should display:

```
DISK FAILURE DETECTED

Failed Disk: Disk 2
```

```
Data Availability: YES

Recovery:
Reconstruct missing data using parity.
```

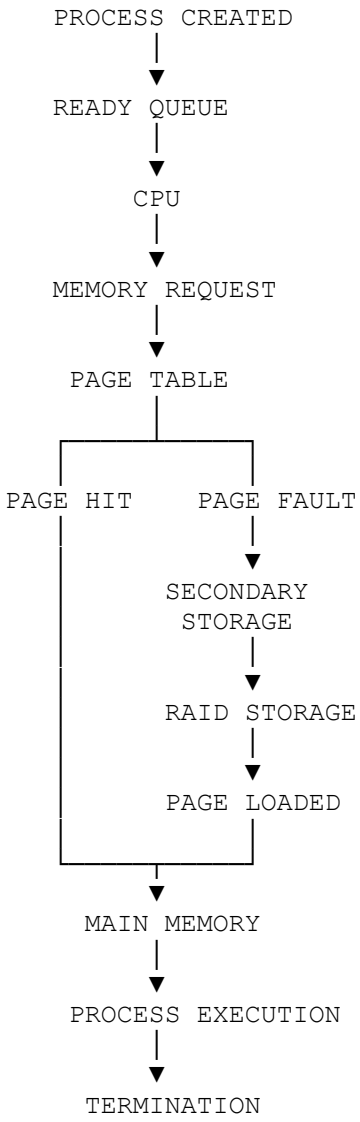
Students should compare RAID levels according to:

- Read performance.
- Write performance.
- Storage efficiency.
- Fault tolerance.
- Recovery capability.
- Number of disks required.

12. Integrated Simulation

The strongest part of the project is the integration of all components.

The final simulator should demonstrate the following sequence:



The simulator should make it possible to observe this process dynamically.

13. User Interface Requirements

The simulator should have a suitable user interface.

The interface should be provided, at minimum:

Process Panel

Display:

- Process ID
- Arrival time
- Burst time
- Priority
- Status
- Memory requirement

CPU Panel

Display:

- Current process
- CPU status
- Current time
- Remaining burst time
- Scheduling algorithm

Memory Panel

Display:

```
Frame 0 | P1-Page 2
Frame 1 | P3-Page 0
Frame 2 | FREE
Frame 3 | P2-Page 1
...
```

Page Table Panel

Display:

- Process
- Page
- Frame
- Valid bit
- Dirty bit
- Location

Virtual Memory Panel

Display:

- Virtual pages
- Page faults
- Page hits
- Replacement algorithm
- Page replacement history

RAID Panel

Display:

- Disk status
- RAID level
- Data distribution
- Parity
- Failed disks
- Recovery status

14. Simulation Controls

The user should be able to control the simulation.

Recommended controls include:

```
[Start]
[Pause]
[Step]
[Reset]
[Generate Processes]
[Generate Page References]
[Disk Failure]
[Run Simulation]
```

The **Step** function is particularly important because it allows the lecturer to see the operating system execute one event at a time.

For example:

```
Step 1:
P1 enters Ready Queue

Step 2:
P1 allocated to CPU

Step 3:
P1 requests Page 3

Step 4:
Page 3 causes PAGE FAULT
```

```
Step 5:
Page 3 loaded from secondary storage

Step 6:
P1 continues execution
```

15. Performance Dashboard

The simulator must provide performance measurements.

At minimum, students should calculate:

CPU

- CPU utilisation
- Average waiting time
- Average turnaround time
- Average response time

Memory

- Number of page faults
- Number of page hits
- Page fault rate
- Page hit rate
- Number of page replacements
- Memory utilisation

Virtual Memory

- Pages in RAM
- Pages on secondary storage
- Page transfer count
- Replacement count

RAID

- Read operations
- Write operations
- Failed disks
- Recovery operations
- Storage utilisation
- Estimated performance

16. Required Experiments

Students must conduct experiments using their simulator.

Experiment 1 – CPU Scheduling

Run at least five processes using two scheduling algorithms.

Compare:

- Waiting time
- Turnaround time
- Response time
- CPU utilisation

Experiment 2 – Memory Size

Run the same workload using different memory sizes.

For example:

```
Experiment A: 8 frames
Experiment B: 16 frames
Experiment C: 32 frames
```

Determine how memory size affects page faults.

Experiment 3 – Page Replacement

Use the same page-reference string with:

- FIFO
- LRU
- Optimal

Compare the number of page faults.

Experiment 4 – Virtual Memory

Compare:

```
Physical Memory = 16 KB
Virtual Memory  = 64 KB
```

and explain how virtual memory allows processes to execute despite insufficient physical memory.

Experiment 5 – RAID

Compare at least three RAID configurations.

Measure or estimate:

- Storage capacity

- Read performance
- Write performance
- Fault tolerance
- Recovery

Students must recommend the most appropriate RAID level for the university's workload and justify their recommendation.

17. Required Scenario

Students must include the following scenario in their final demonstration.

Scenario: Large Research Process

A research application requires:

```
Process size:        48 KB
Physical memory:     16 KB
Virtual memory:      64 KB
Page size:            4 KB
Number of disks:      4
```

The application generates the following page references:

```
0, 1, 2, 3, 4, 5, 0, 1, 6, 7,
2, 3, 8, 9, 0, 1, 4, 5, 10, 11
```

Students must demonstrate:

1. How the process is divided into pages.
2. How pages are allocated to frames.
3. What happens when the process references a page not in RAM.
4. How a page fault occurs.
5. How the operating system selects a victim page.
6. How the victim page is moved to secondary storage.
7. How the requested page is loaded into RAM.
8. How the page table is updated.
9. How virtual memory enables execution.
10. How secondary storage is organised using RAID.
11. What happens when one RAID disk fails.
12. How the system recovers from failure.

18. Technical Report

Students must submit a technical report containing:

Chapter 1 – Introduction

Explain:

- Problem being addressed.
- Purpose of the simulator.
- Objectives.
- Scope.

Chapter 2 – Operating Systems Concepts

Discuss:

- Process management.
- CPU scheduling.
- Main memory.
- Paging.
- Virtual memory.
- Page replacement.
- Secondary storage.
- RAID.

Chapter 3 – System Design

Include:

- System architecture.
- Flowcharts.
- Class diagrams where appropriate.
- Data structures.
- Process model.
- Memory model.
- Page table design.
- RAID model.

Chapter 4 – Implementation

Explain:

- Development environment.
- Algorithms implemented.
- Important implementation decisions.
- User interface.
- Simulation engine.

Chapter 5 – Experiments and Results

Present:

- CPU scheduling results.
- Memory results.
- Page replacement results.

- Virtual memory results.
- RAID results.

Use tables and graphs where appropriate.

Chapter 6 – Discussion

Students must explain:

- Why do page faults occur.
- Why do some replacement algorithms perform better.
- How memory size affects performance.
- How virtual memory affects execution.
- Why do RAID levels differ.
- Which RAID level is recommended and why.

Chapter 7 – Conclusion

Summarise:

- What was achieved.
- Limitations.
- Lessons learned.
- Possible improvements.

19. Demonstration Requirements

Each student/group must demonstrate the working simulator.

During the demonstration, the lecturer may ask students to modify:

- RAM size.
- Page size.
- Number of processes.
- CPU scheduling algorithm.
- Page replacement algorithm.
- Virtual memory size.
- RAID level.
- Number of disks.
- Disk failure conditions.

Students must be able to explain the resulting changes.

A student who can run the simulator but cannot explain the underlying operating-system concepts should not receive full marks.

20. Assessment Rubric – 100 Marks

Component	Marks
Process creation and CPU execution	10
CPU scheduling and performance analysis	10
Paging and page-table implementation	15
Page faults and movement between memory levels	10
Virtual memory implementation	10
Page replacement algorithms and comparison	10
RAID simulation and disk failure	10
Integrated simulation	10
User interface and visualization	5
Experiments, results and analysis	5
Technical report and documentation	5
TOTAL	**100**

21. Final Deliverables

Each group must submit:

1. **Working simulation application**
2. **Source code**
3. **Technical report**
4. **User manual**
5. **Test data**
6. **Experimental results**
7. **Presentation/demo**
8. **README/instructions for running the application**

The source code must be sufficiently documented to allow me to understand the implementation.